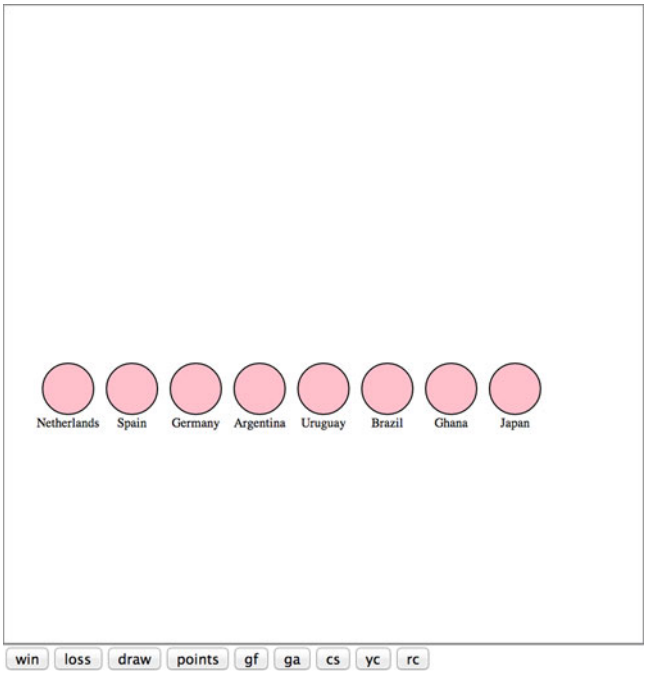


Figure 3.3 Buttons for each numerical attribute are appended to the `controls` div behind the `viz` div. When a button is clicked, the code runs `buttonClick`.

We use `d3.keys` and pass it one of the objects from our array. The `d3.keys` function returns the names of the attributes of an object as an array. We've filtered this array to remove the `team` and `region` attributes because these have nonnumerical data and won't be suitable for the `buttonClick` functionality we define. Obviously, in a larger or more complex system, we'll want to have more robust methods for designating attributes than listing them by hand like this. You'll see that later when we deal with more complex datasets. In this case, we bind this filtered array to a selection to create buttons for all the remaining attributes, and give the buttons labels for each of the attributes, as shown in figure 3.3.

The `.on` function is a wrapper for the traditional HTML mouse events, and accepts `"click"`, `"mouseover"`, `"mouseout"`, and so on. We can also access those same events using `.attr`, for example, using `.attr("onclick", "console.log('click')")`, but notice that we're passing a string in the same way we would using traditional HTML. There's a D3-specific reason to use the `.on` function: it sends the bound data to the function automatically and in the same format as the anonymous inline functions we've been using to set style and attribute.

We can create buttons based on the attributes of the data and dynamically measure the data based on the attribute bound to the button. Then we can resize the circles representing each team to reflect the teams with the highest and lowest values in each category, as shown in figure 3.4.

We can use `.on()` to tie events to any object, so let's add interactivity to the circles by having them indicate whether teams are in the same FIFA region: